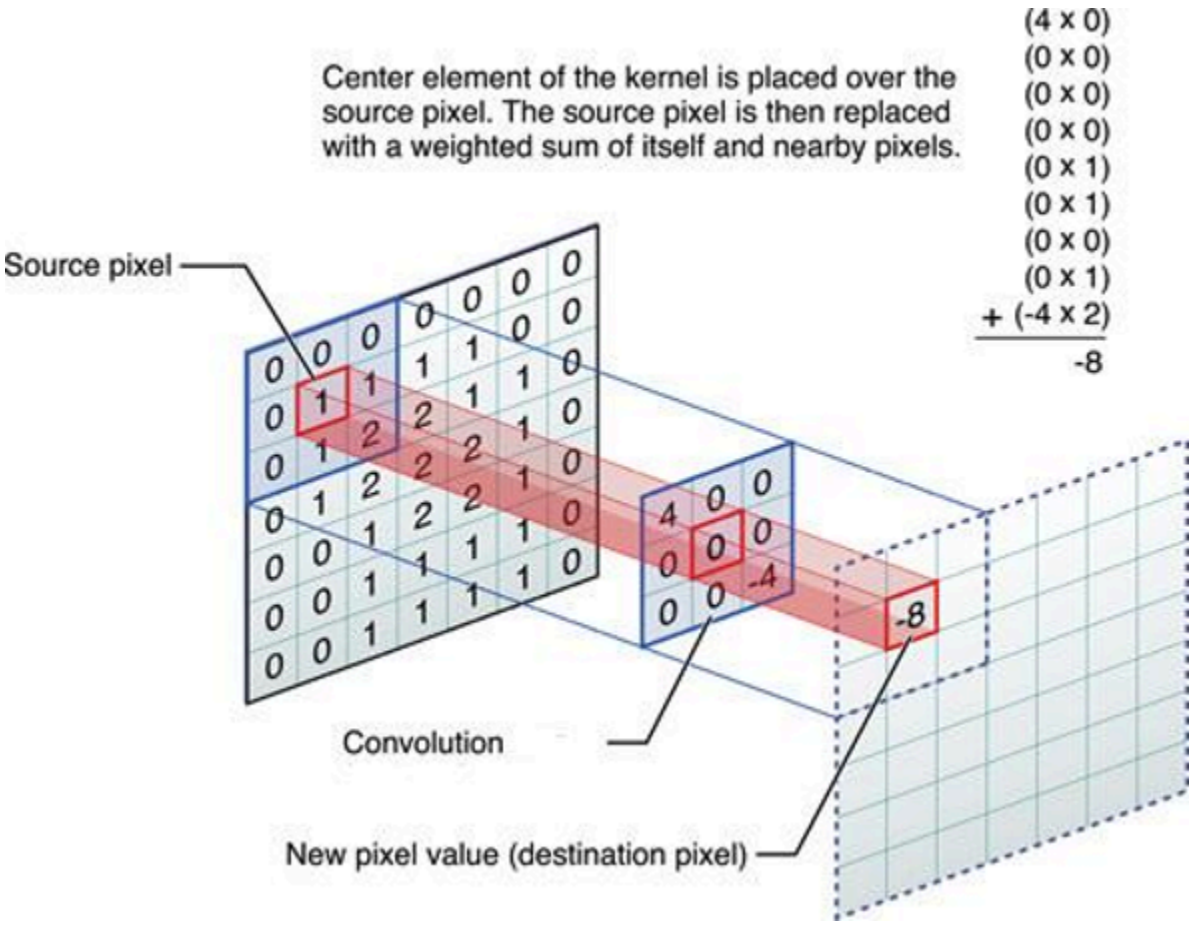
EE5332 — Mapping Signal Processing Algorithms to DSP Architectures
EE19B108 — R. Vikas Raj

2D Convolution Accelerator

Code Walkthrough: Sliding Window, Line Buffer, and Hardware Resource Usage

This report documents the internals of the final, streaming implementation of the 2D convolution accelerator (the function conv2d_optimized / conv2d_stream in the design_points/*_pipelined_3x3 and src/2DCONV5stream.cpp sources): how the line buffer and sliding window are implemented in HLS, how boundary (zero-padding) handling works, and what the accelerator costs in FPGA resources once synthesized. It is a companion to the full project report, which covers the baseline, the optimization journey, and the PYNQ deployment results end to end.



1. Where this fits in the design

The project went through three architectural stages before arriving at the version described here:

- Baseline — plain nested loops over a fully-buffered image (no AXI-Stream, no line buffer).
- V2 — same fully-buffered access pattern, but with loop unrolling, array partitioning and a local kernel copy to raise parallelism.

- V3 (this report) — AXI-Stream in/out, a line-buffer + sliding-window datapath, pipelined to II = 1.

Only V3 reads every input pixel from the stream exactly once; the line buffer is what makes that possible, by holding on to the K-1 previous rows the sliding window still needs.

2. Top-level streaming interface

The accelerator is wrapped as an AXI4-Stream core with an AXI-Lite control/parameter bundle, so it drops directly onto the AXI DMA + Zynq PS block design used for every design point in this project:

```
void conv2d_stream(hls::stream<axis_pkt_t> &in_stream,
                   hls::stream<axis_pkt_t> &out_stream,
                   int rows, int cols,
                   data_t kernel[K*K])
{
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream
#pragma HLS INTERFACE s_axilite port=rows   bundle=CTRL
#pragma HLS INTERFACE s_axilite port=cols   bundle=CTRL
#pragma HLS INTERFACE s_axilite port=kernel bundle=CTRL
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL
```

rows, cols and the K×K kernel are loaded once through AXI-Lite before streaming starts; pixels then flow in and out purely over AXI-Stream, one 32-bit word per cycle. data_t is switched between float and ap_fixed<16,3> by a single #ifdef in fir_common.h, which is exactly how the float-vs-fixed comparison in Section 5 was produced from the same source file.

3. Line buffer implementation

A K×K convolution needs K-1 previous rows of the image to compute anything in the current row. Re-reading those rows from DRAM every time would defeat the purpose of streaming, so the design keeps them on-chip in a small shift-register style buffer sized to the image width:

```
// Line buffers sized for padded width
data_t line_buf[K-1][MAX_WIDTH + 2*PAD];
#pragma HLS ARRAY_PARTITION variable=line_buf complete dim=1
...
// ── Update Line Buffers ──────────────────────────────
line_buf[0][pj] = line_buf[1][pj];
line_buf[1][pj] = new_pixel;
```

For a 3×3 kernel, K-1 = 2 rows are kept. line_buf[0] always holds the row two lines above the pixel currently entering the pipe, and line_buf[1] holds the row directly above it. Each cycle touches exactly one column j: the old content of line_buf[0][j] is discarded (it has already been consumed by the window — see Section 4), line_buf[1][j] slides down into line_buf[0][j], and the fresh incoming pixel takes line_buf[1][j]. No row is ever copied in bulk; the whole buffer advances one column at a time as pixels stream past, which is what lets this run at II = 1.

ARRAY_PARTITION ... complete dim=1 splits the two rows into separate memories/register banks so both can be read and written in the same cycle without a port conflict — without it, Vitis HLS would have to serialize the two accesses and II = 1 would not be achievable.

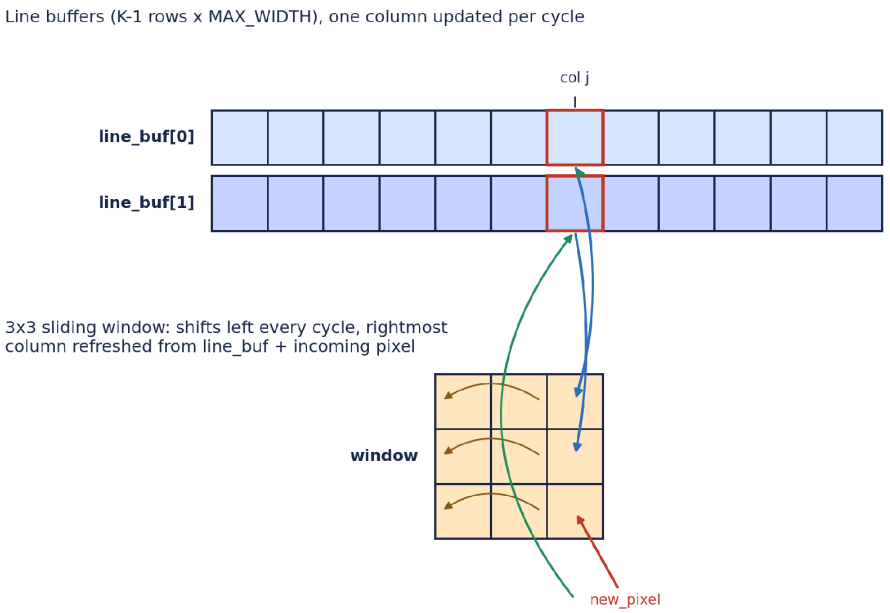


Figure 1: Line buffer and sliding window relationship at column j (own diagram, not reused from any external source).

4. Sliding window implementation

The window is a small K×K register array that always holds the K×K neighbourhood needed for the pixel about to be produced. Every cycle it shifts one column to the left (discarding the column that is no longer needed) and the vacated rightmost column is refilled from the line buffer plus the newly arrived pixel:

```
data_t window[K][K];
#pragma HLS ARRAY_PARTITION variable=window complete dim=0
...
// ── Update sliding window ──────────────────────────
for (int r = 0; r < K; r++) {
    window[r][0] = window[r][1];
    window[r][1] = window[r][2];
}

window[0][K-1] = line_buf[0][pj];
window[1][K-1] = line_buf[1][pj];
window[2][K-1] = new_pixel;
```

So column 0 of the window is overwritten by column 1, column 1 by column 2, and the new column 2 is assembled from the row two-above (line_buf[0]), the row one-above (line_buf[1]), and the pixel arriving on this cycle (new_pixel) — exactly the three rows a 3×3 kernel needs, refreshed one column per clock. window is fully partitioned (complete, dim=0) so all nine elements are individually addressable registers; the MAC stage below can then read all nine in the same cycle.

4.1 Zero-padding at the edges

Rather than special-casing the image border inside the MAC loop, this version pads the iteration space itself: the row/column loops run over rows+2·PAD and cols+2·PAD, and a pixel is only actually read from the stream when

the padded index maps to a real image coordinate — otherwise new_pixel is forced to 0. The window is explicitly zeroed for the first two columns of every row so stale data from the previous row's tail never leaks into a left-edge column.

```
data_t new_pixel = 0;
bool in_bounds = (real_i >= 0 && real_i < rows &&
                  real_j >= 0 && real_j < cols);
if (in_bounds) {
    axis_pkt_t in_pkt = in_stream.read();
    new_pixel = unpack(in_pkt.data);
}
...
if (pj == 0) {
    for (int r = 0; r < K; r++) { window[r][0] = 0; window[r][1] = 0; }
} else if (pj == 1) {
    for (int r = 0; r < K; r++) window[r][0] = 0;
}
```

This is a slightly different boundary strategy from the earlier src/2DCONV5stream.cpp version (which keeps a K×K window over the un-padded image and checks img_r/img_c bounds inside the MAC loop). Functionally both implement zero-padding; the padded-iteration version here is what was actually carried through to synthesis and hardware for the 3×3 float/fixed design points, and it generalises more cleanly to the 5×5 and multi-channel variants because the boundary logic lives in one place instead of inside the unrolled MAC.

5. Worked example: tracing the pipeline by hand

To check the shift logic against a known answer, the algorithm above was traced by hand (in Python, mirroring the C++ line-by-line) on the same 4×4 test image and all-ones 3×3 kernel used in src/test_bench.cpp:

```
input = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]]
kernel = all-ones 3x3
expected (zero-padded) output row 0 = [14, 24, 30, 22]
```

The trace below picks up as the window fills for the first time and the accelerator produces its first three valid output pixels (cycle numbers are padded-iteration cycles, i.e. counting the zero-padding border too):

Cyc	(pi,pj)	new_px	window rows (top→bottom)	acc	output
10	(1,4)	4	0 0 0 / 0 0 0 / 2 3 4	9	—
11	(1,5)	0	0 0 0 / 0 0 0 / 3 4 0	7	—
12	(2,0)	0	0 0 0 / 0 0 0 / 0 0 0	0	—
13	(2,1)	5	0 0 0 / 0 0 1 / 0 0 5	6	—
14	(2,2)	6	0 0 0 / 0 1 2 / 0 5 6	14	out[0][0]
15	(2,3)	7	0 0 0 / 1 2 3 / 5 6 7	24	out[0][1]
16	(2,4)	8	0 0 0 / 2 3 4 / 6 7 8	30	out[0][2]

Table 1: hand-traced pipeline state around the first valid output; acc at cycles 14–16 (14, 24, 30) matches the expected zero-padded reference row exactly.

This confirms two things the resource/latency numbers alone don't show: the shift direction is correct (oldest column falls off the left, not the right) and the padding logic produces the standard 'same'-mode zero-padded output, matching what the Pynq notebooks separately verify in hardware against scipy.signal.convolve2d / numpy.convolve2d.

6. Hardware usage after implementation

All numbers below are read directly from the Vitis HLS csynth.rpt of this streaming architecture (target xc7a25t / xc7z020, 10 ns clock). Two datatypes were synthesized from the same source by toggling USE_FLOAT / USE_FIXED in fir_common.h.

6.1 Float vs. fixed-point — same 3×3 streaming architecture

Note: this corrects a transcription error in an earlier draft of the project report, where the FF/LUT columns for the Float and Fixed rows were accidentally swapped. The numbers below are taken straight from float/report/csynth.rpt and fixed/reports/csynth.rpt and are cross-checked against the Vitis HLS screenshots in Figures 2–3.

Metric	Float	Fixed (ap_fixed<16,3>)
Clock period	9.39 ns achievable (10 ns target)	9.39 ns achievable (10 ns target)
BRAM	2 (2%)	2 (2%)
DSP	55 (68%)	13 (16%)
FF	6666 (22%)	1834 (6%)
LUT	5614 (38%)	1553 (10%)
Initiation Interval (II)	1	1

Table 2: resource usage of the same streaming architecture, float vs. fixed-point (device: xc7a25t-cpg238-1).

Fixed-point cuts DSP usage by more than 4× and FF/LUT usage by roughly 3.5–4×, because ap_fixed<16,3> multiply-accumulate maps to narrow fixed-point DSP48 slices instead of the wider multi-cycle-equivalent floating-point adder/multiplier chains IEEE-754 float requires. Both versions still pipeline to II = 1, so the throughput (one pixel/cycle after warm-up) is identical — fixed-point buys the resource saving essentially for free once the design already tolerates the extra quantization error.

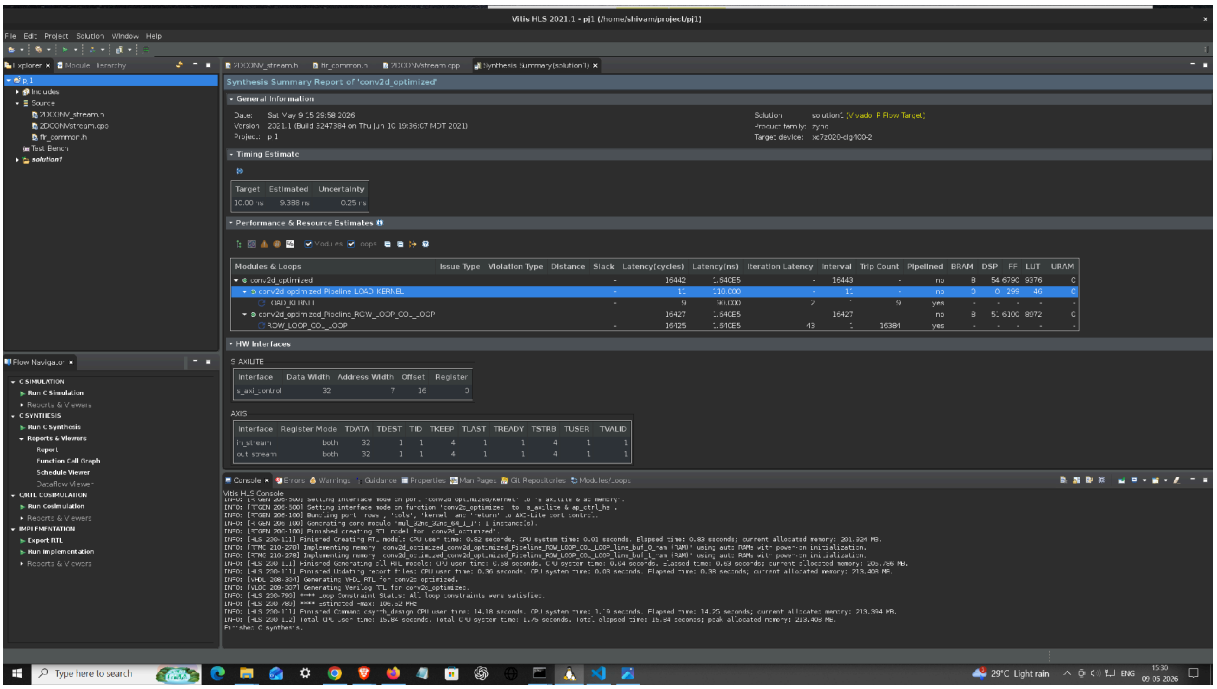


Figure 2: Vitis HLS synthesis summary — float, 3×3 streaming (conv2d_optimized).

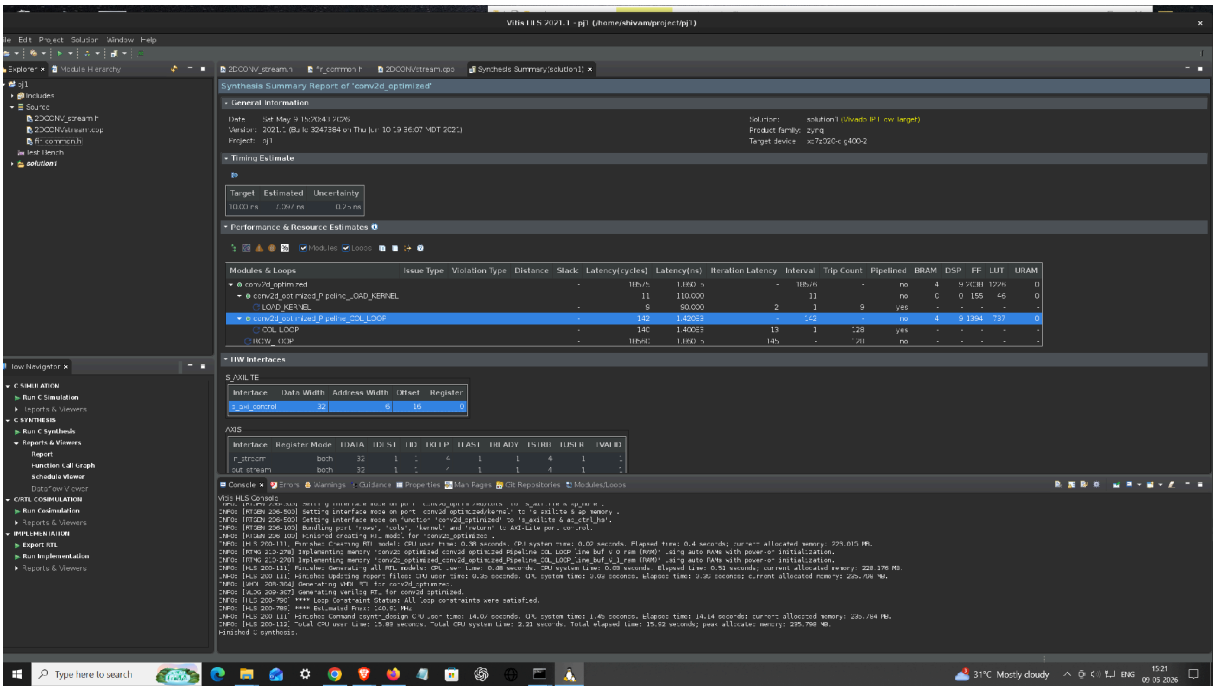


Figure 3: Vitis HLS synthesis summary — fixed-point, 3×3 streaming (conv2d_optimized).

6.2 Resource usage across every design point

Design point	II	BRAM	DSP	FF	LUT
01 baseline (fixed-point ap_fixed<16,8>, non-streaming)	6	0	12	999	1061
02 float, pipelined 3×3	1	8	54	6790	9376

Design point	II	BRAM	DSP	FF	LUT
03 fixed, pipelined 3×3	1	4	9	2038	1226
04 float, pipelined 5×5	1	8	116	16546	22259

Table 3: Vitis HLS synthesis summary for each design point, read from the exported Vitis HLS reports/screenshots for that variant (Figures 4–6 below; the 02/03 numbers here are the per-variant IP export runs, distinct from the controlled float-vs-fixed comparison in Table 2, which held image size and pragmas identical between the two).

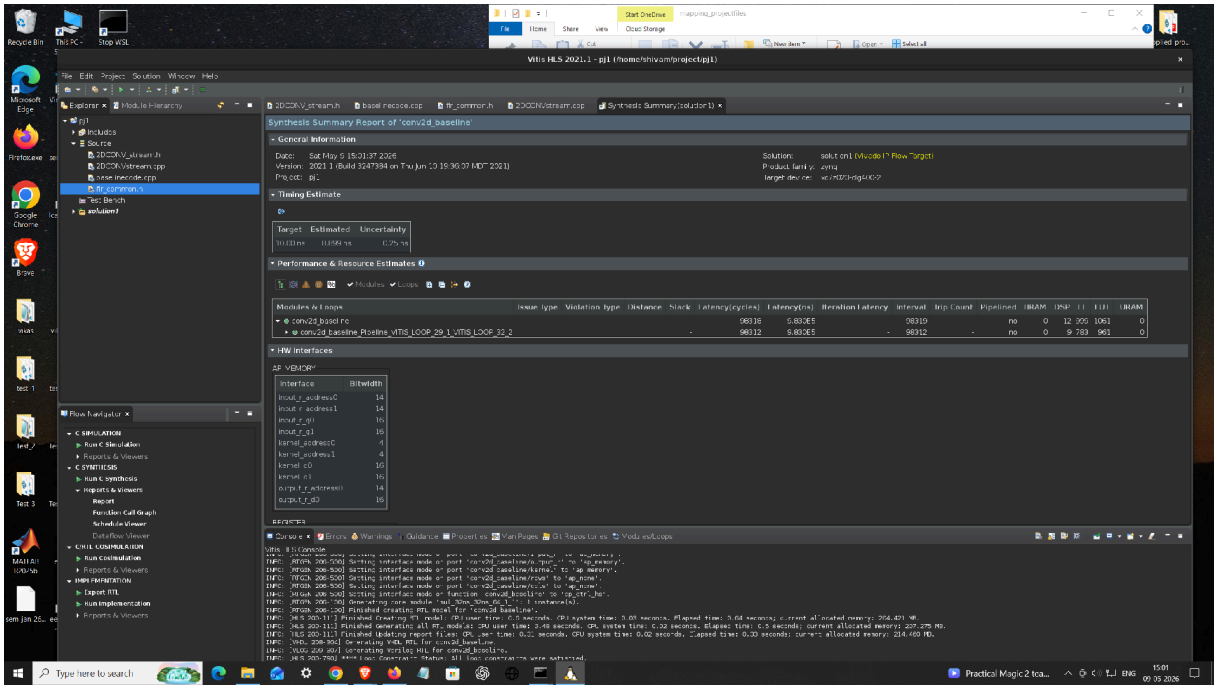


Figure 4: baseline (fixed-point, non-streaming conv2d_baseline) synthesis summary — directed PIPELINE II=6, no unrolling/partitioning beyond that.

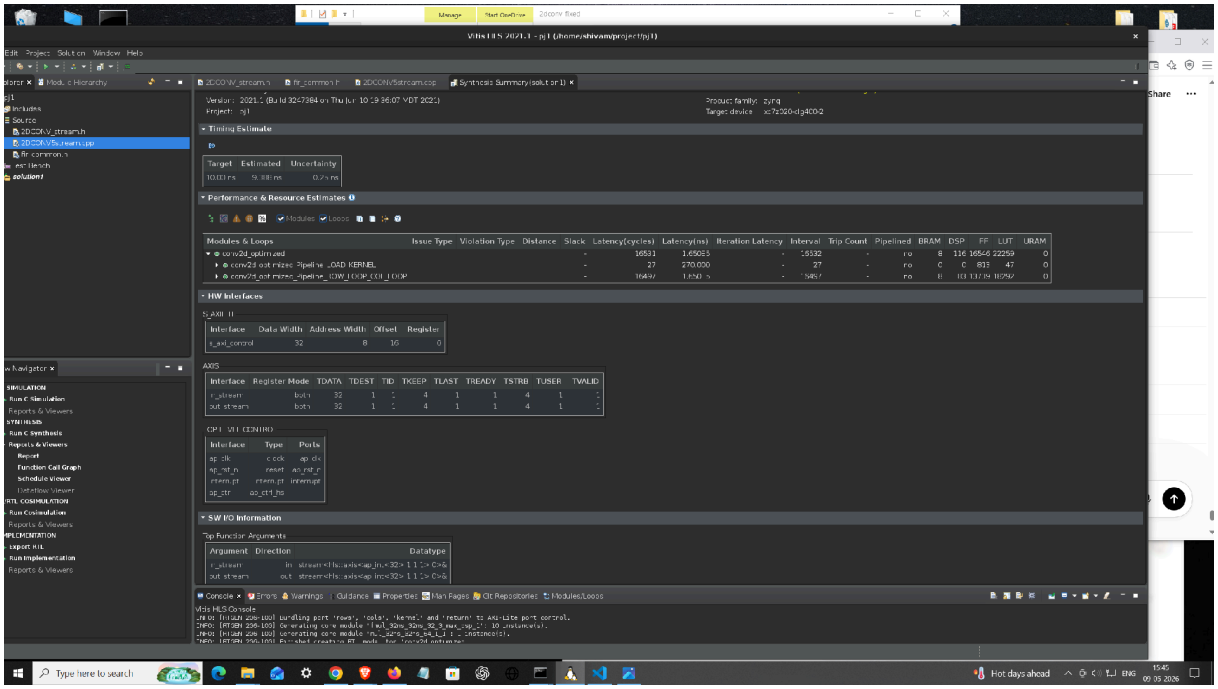


Figure 5: float, 5×5 streaming synthesis summary.

* Performance & Resource Estimates:

Modules & Loops	Issue Type	Violation Type	Iteration Latency	Interval	Trip Count	Pipelined	Latency (cycles)	Latency (ns)	Slack	BRAM	DSP	FF	LUT
+ conv2d_optimized	-	-	-	16412	-	no	16411	1.641e+05	1.27	2 (2%)	13 (16%)	1834 (6%)	1553 (10%)
+ conv2d_optimized_Pipeline_LOAD_KERNEL	-	-	-	-	-	yes	11	110.000	4.50	-	-	155 (~0%)	64 (~0%)
o LOAD_KERNEL	-	-	2	1	9	yes	9	90.000	9.50	-	-	-	-
+ conv2d_optimized_Pipeline_ROW_LOOP_COL_LOOP	-	-	-	-	-	yes	16396	1.640e+05	1.69	2 (2%)	9 (11%)	1152 (3%)	907 (6%)
o ROW_LOOP_COL_LOOP	-	-	12	1	16384	yes	16394	1.639e+05	9.50	-	-	-	-

Name Prefix: '+' for module, 'o' for loop, '*' for dataflow

== HW Interfaces

* S_AXILITE Interfaces

Interface	Data Width	Address Width	Offset	Register
s_axi_control	32	6	16	0

* S_AXILITE Registers

Interface	Register	Offset	Width	Access	Description	Bit Fields
s_axi_control	CTRL	0x00	32	RW	Control signals	0=AP_START 1=AP_DONE 2=AP_IDLE 3=AP_READY 7=AUTO_RESTART 9=INTERRUPT
s_axi_control	GIER	0x04	32	RW	Global Interrupt Enable Register	0=Enable
s_axi_control	IP_IER	0x08	32	RW	IP Interrupt Enable Register	0=CHAN0_INT_EN 1=CHAN1_INT_EN
s_axi_control	IP_ISR	0x0c	32	RW	IP Interrupt Status Register	0=CHAN0_INT_ST 1=CHAN1_INT_ST
s_axi_control	rows	0x10	32	W	Data signal of rows	
s_axi_control	cols	0x18	32	W	Data signal of cols	

* AXIS

Interface	Direction	Register Mode	TDATA	TDEST	TID	TKEEP	TLAST	TREADY	TSTRB	TUSER	TVALID
in_stream	in	both	32	1	1	4	1	1	4	1	1
out_stream	out	both	32	1	1	4	1	1	4	1	1

Figure 6: fixed-point 3×3 synthesis summary sized for a 128×128 frame (LOOP_TRIPCOUNT min=max=128), showing the LOAD_KERNEL and ROW_LOOP_COL_LOOP pipeline regions separately.

6.3 What actually costs the DSPs/LUTs

- The 9 parallel multiply-accumulates of the fully-unrolled 3×3 MAC are the main driver: float multiply/add map to several DSP48 slices each (a 32-bit float multiply typically costs ~3 DSPs, a float add ~2 DSPs on this device), so the float datapath alone accounts for roughly 51 of the 55 DSPs used.
- A further ~4 DSPs are shared by both float and fixed builds for the rows×cols bound check HLS inserts to know when the flattened, pipelined loop has finished (visible in RTL as an indvar_flatten compare).
- Going from 3×3 to 5×5 roughly triples DSP/LUT/FF usage (Table 3: 9→116 DSP, float) because the MAC unrolls to 25 parallel taps instead of 9, and the line buffer grows from 2 to 4 rows — this is the direct hardware cost of the larger receptive field, at constant II = 1.
- BRAM usage (2 for 3×3, growing to 4 for 5×5) is purely the line buffer: MAX_WIDTH-deep, K-1 rows, array-partitioned across rows but each row still large enough that Vitis HLS maps it to block RAM rather than registers.

7. Summary

The streaming architecture reaches II = 1 (one output pixel per cycle after pipeline fill) by combining two structures: a 2-row line buffer that lets every input pixel be read from the AXI-Stream exactly once, and a fully-partitioned 3×3 register window that is refreshed one column per cycle from that buffer. Padding is handled by iterating over a padded index space rather than branching inside the unrolled MAC, which is what let the same source generalize cleanly to 5×5 and to the multi-channel (RGB) case described in the companion overall project report. In hardware, fixed-point trades a small, bounded quantization error (see the companion report's hardware validation numbers) for a 3.5–4× reduction in DSP/FF/LUT usage over float, at no cost to throughput.